Distributed Compute Cluster: Team Masterplan

Team of 3 · 400 marks · 25% of each person's course grade · Deadline: 2 December 2026

1. How we split it (and why)

Three ways to divide the project were possible:

- **Three whole milestones (A does Milestone 1, B does Milestone 2, and so on):** rejected. Milestone 2 cannot be tested until Milestone 1 runs, so two people would sit waiting.
- **One program each (client / scheduler / worker):** rejected. The scheduler alone is 105+ marks and the client is 25, so it would be badly unequal.
- **Chosen: every section is split by ownership of files, after agreeing one shared contract.** Each person owns separate files across the whole project, builds against stubs and small mock programs, and never needs another person's code to be finished before starting.

Three ideas make 'nobody waits' real:

1. **Contract first.** Before anyone codes, all three agree on and freeze the data structures, message types and function signatures (Section 4). This is the only part done together.
2. **Stubs.** Every function someone else will write exists from day one as an empty stub, so the whole project compiles from the first hour.
3. **Mocks.** Each person writes a tiny fake of whatever they would otherwise wait for (a fake scheduler, a fake worker, a fake client). 50 to 100 lines each, and well worth it.

2. The split at a glance

Person	Role	Sections (marks)	Total
A	Worker & Sandbox	2B (10), 5A (30), 5B (10), 5C (10), 5D (10), 5E (10), 9 (10), 12 (15), 14 (30)	135
B	Scheduler Core	2A (5), 4A (30), 4B (25), 4C (40), 4D (10), 10 (20)	130
C	Client & Scheduler Extensions	2C (10), 3 Login (10), 3 Commands/flags/events (15), 6 (20), 7 (10), 8 (20), 11 (15), 13 (35)	135

Total: 400. The 5-mark gap is deliberate: B's sections (concurrency, fault tolerance) are the hardest to debug, so B carries 5 fewer marks. Decide who is A, B and C by preference or a coin toss at Gate 0.

3. Who owns what

Repo layout, with the owner of each file group:

```
common/      protocol.h types.h log.c net.c discovery.c hash.c        (C
writes; contract rules apply)
scheduler/   main.c queue.c workers.c pass.c heartbeat.c state.c preempt.c
(B)
             events.c logbuf.c deps.c quota.c filesend.c
(C)
             cron.c migration.c
(A)
worker/      main.c sandbox.c output.c telemetry.c crash.c drain.c
checkpoint.c   (A)
             cache.c
(C)
client/      main.c login.c commands.c cmd_logs.c
(C)
             cmd_cron.c cmd_migrate.c
(A)
tests/       mocks/a  mocks/b  mocks/c  demo/
Makefile     wildcard rules, so adding a file never needs a Makefile edit
```

Golden rule: you may change anything in your own files without asking anyone. Touching someone else's file or the contract goes through Section 6.

Person A: Worker & Sandbox (135)

- Owns everything in worker/ except cache.c, plus scheduler/cron.c, scheduler/migration.c, client/cmd_cron.c and client/cmd_migrate.c.
- Also owns the worker side of heartbeats: sending one every 2 seconds with CPU load, running-job count and per-job telemetry (B reads these on the scheduler side).
- Needs from others (stubbed until real): discovery from C (until then use the fallback address), log_line() from C (in the Stage 0 skeleton), cache_prepare() from C.
- Mock: a fake scheduler that accepts a worker, sends JOB_ASSIGN, and prints every update it receives.

Person B: Scheduler Core (130)

- Owns the scheduler's connection loop, the message dispatch table, job_set_state() and the scheduling pass.
- Writes the pass in exactly the order of the brief's 'Order of Scheduling Decisions' page, calling hook stubs for deps, quota, cache, checkpoint and binary sending. When owners finish their hooks, the pass picks them up with no change from B.
- Message handlers stay thin: each one just calls the function owned by that section's owner (for example an output chunk goes to C's logbuf_append).
- Also owns scheduler-side cancel (QUEUED removal, kill command for RUNNING, WAITING).
- Mock: a fake worker and fake client (script or small C program) that register, heartbeat, stop heartbeating, and submit jobs.

Person C: Client & Scheduler Extensions (135)

- Owns client/ (except A's two command files), common/, scheduler/events.c, logbuf.c, deps.c, quota.c, filesend.c, and worker/cache.c.
- Parses ALL submit flags from day one, including later ones (-depends-on, -group, -preemptible, -every), into the full job struct, so later sections need no client changes.
- Delivers early because everyone uses them: log_line() (during Stage 0), the client command table, and discovery.
- Mock: a fake scheduler that accepts login and commands and pushes events. Sections 8, 11 and 13 are tested with unit tests on hand-made job and worker structs, so they need no running cluster.

4. The contract (written together once, then frozen)

Hooks: who writes which function

Function	Written by	Called by	What it does
job_set_state(job, state, reason)	B	everyone	The only way to change a job's state: logs it, sends the client event, updates timestamps
log_line(tag, fmt, ...)	C	everyone	The one central logging function

Function	Written by	Called by	What it does
emit_event(user, fmt, ...)	C	A, B	Pushes an [Event] line to that user's client(s)
deps_on_submit(job), deps_on_finish(job)	C	B	WAITING to QUEUED, cascade CANCELLED, reject unknown ids
quota_should_skip(job)	C	B's pass	Fair-share check between pass step 1 and 2
cache_worker_has(worker, hash)	C	B's pass	Tie-break: binary already cached
logbuf_append(job, bytes)	C	B's output handler	64 KB buffer and forwarding to followers
send_checkpoint(job, worker), then send_binary(job, worker)	A, then C	B after choosing a worker	Called in this order
cron_tick()	A	B's loop, once per second	Spawns due cron copies
migrate_request(job)	A	client command, via scheduler	Starts a migration
KILL message handling on worker	A	B (cancel, preempt, migrate)	SIGKILL / SIGTERM handling and cleanup

Write these down in Stage 0

- Enums: all 8 job states and all worker states (ALIVE, SUSPECTED, DEAD, DRAINING), even the ones not needed yet.
- The full job struct: id, user, binary path, hash, cores, ram, time, prio, gpu, depends list, group, preemptible, every, attempts, assigned worker, excluded worker, latest checkpoint (data, number, size, time), telemetry, timestamps.
- The worker struct: id, state, total and free cores/RAM/GPU, CPU load, running jobs, missed heartbeats, cached hashes.

- Message framing (suggestion: fixed header with type and payload length, then payload) and the message list: REGISTER, REGISTER_ACK, HEARTBEAT, JOB_ASSIGN, JOB_UPDATE (state plus reason text), JOB_OUTPUT, CHECKPOINT, KILL, DRAIN, CACHE_REPORT, FILE_DATA, CLIENT_LOGIN, CLIENT_CMD, EVENT.
- Constants: retry limit (3), heartbeat interval (2 s), cache env var names.
- Exact binary names and command lines from Section 2 of the brief.
- Text conventions: uppercase log tags, event line format.

5. Overall order of work

Stage 0: Contract and skeleton (all three, together, one sitting)

Create the repo, add all three as collaborators, and set git user.name and email correctly on every machine. Fill in Section 4, write the headers and empty stubs, the Makefile and the mock folders. Confirm `make` produces ./scheduler, ./worker and ./client. **Freeze** when each person can honestly say: 'I can build my whole part without asking anyone anything.'

Stage 1: Milestone 1 pieces, in parallel

- **A:** worker CLI and clamping, then registration and heartbeat sender, then 5A sandbox, then 5B output capture.
- **B:** scheduler CLI and listener, then concurrency model with bounded queue and worker table, then the FIFO/PRIORITY pass.
- **C:** client login, prompt, commands, flags and events, then discovery (both the UDP responder and the finder).
- **Gate 1 (together):** one scheduler, one worker, one client work end to end.

Stage 2: Milestone 2 pieces, in parallel

- **A:** 5C cancel, then 5D telemetry, then 5E crash diagnosis, then 9 draining.
- **B:** 4C heartbeats and rescue, then 4D load-aware choice.
- **C:** 6 streaming logs, then straight into Stage 3 work (C has the heaviest Milestone 3 load).
- **Gate 2 (together):** failure, cancel, drain and streaming demo, plus the rebalance check (Section 7).

Stage 3: Milestone 3 pieces, in parallel

- **A:** 12 cron (small), then start 14 early since A has the lightest Milestone 3.
- **B:** 10 preemption.

- **C:** 8 dependencies, then 11 quotas, then 13 caching. These are unit-testable alone, so C starts as soon as the client is done and does not wait for Gate 2.
- **Gate 3 (together):** dependencies, preemption, quota, cron and caching demo.

Stage 4: Milestone 4 and hardening

- **A:** finish 14 (checkpoints and migrate).
- **B and C:** relief duties below, then bug-hunting in the whole system.
- **Gate 4 (together):** migration demo, then the full final checks.

Gates are not waiting rooms. Start the next stage on your own branch before the gate happens. A gate is just a joint check of merged work, about one session, ending with a 5-minute walkthrough from each person.

Relief duties (equal, independent of marks)

- **A:** end-to-end test and demo scripts in tests/demo/, using the brief's examples, run under sudo.
- **B:** README and design notes (concurrency model choice, state machines, protocol summary).
- **C:** fresh-clone build check, Linux and macOS run, and commit-message audit before submission.

6. Git workflow

- **main always builds.** Never push something that breaks `make`.
- **One branch per task**, short-lived: `a/sandbox-limits`, `b/heartbeats`, `c/login`. Merge back at least once every working session.
- **Your own files:** merge to main yourself once `make` passes and your mock test runs. No approval needed, so nobody waits.
- **Someone else's file, anything in common/, or the hooks table:** open a PR, tag the owner, and say so in the group chat. Contract changes must be additive (add a field or message; never rename or remove).
- **Review rotation, after merge, same day:** A reviews B, B reviews C, C reviews A. Fix forward with a new commit.
- **Keep history visible.** Use merge or rebase-merge, not squash, so each person's individual commits stay attributed. The brief says commits show who did what and bad messages are penalised.

- **Commit format:** `<imperative summary> (<section>)`, for example `Add 500 ms output tailing to worker (5B)`. One logical change per commit, commit often, push at the end of every session. Never 'fix', 'changes' or 'asdf'.
- Run `git pull --rebase` before every push.

7. Keeping it equal

- **Rebalance check at Gate 2:** a 5-minute honest check. If someone is clearly behind, move one small item (10 to 15 marks) from the heavier person to the lighter one and update this doc. Only at gates, never mid-stage.
- **Finished early?** Do your relief duty, write tests, or review. Do not silently take over someone's item; offer, and let them say yes.
- **Blocked?** Use the stub and move on. Message the group only if the contract has to change.
- **Everyone must understand everything.** Marks are individual, so each person must be able to explain and defend all of the code. The gate walkthroughs are how.
- **Brief's rules to remember:** AI agents that touch your code are banned (a chat window for explanations, boilerplate and debugging is fine); modular code, no giant main.c; fork() and execvp(), never system(); test with sudo; must run on Linux and macOS with no Docker.

8. Checklists

8.1 Start of every work session

- [ ] `git pull --rebase`, then `make clean && make` passes
- [ ] Read the group chat; check whether the contract changed
- [ ] Pick the next item from your own list and switch to its branch

8.2 Before every commit

- [ ] Builds with `-Wall -Wextra` and no new warnings
- [ ] One logical change, message in the agreed format
- [ ] No debug prints, hardcoded paths or IPs; return values of system calls are checked
- [ ] `git diff --stat` shows only your own files

8.3 Before merging to main

- [ ] `make clean && make` passes

- [ ] Your mock test passes and the last gate demo still works
- [ ] Only your own files touched (otherwise PR and tag the owner)
- [ ] Your stub is replaced by the real function and the stub is deleted

8.4 Whenever the contract changes

- [ ] Additive only (no renames or removals)
- [ ] Exact change posted in the group chat
- [ ] Everyone has pulled before continuing
- [ ] This doc updated

8.5 Person A checklist

- [ ] `./worker [<ip> <port>] -cores -ram [-gpu] [-discovery-port]` parses exactly like the brief
- [ ] cores and ram clamped to real values with a warning (sysconf; sysctl on macOS); gpu not clamped
- [ ] Discovery first, fallback message second, error and exit if neither works
- [ ] Registers, logs its ID, heartbeat every 2 s with CPU load, running count and telemetry
- [ ] 5A: /tmp/osn_job_<id> dir, binary copied (skipped for PATH commands), chown to nobody, setgid then setuid with return values checked, RLIMIT_CPU, RLIMIT_AS, RLIMIT_NPROC, all before execvp
- [ ] Not running as root: clear warning, skip chown and privilege drop, keep going
- [ ] Wall-clock timer kills with SIGKILL and reports 'Wall-clock timeout exceeded'; macOS watches RAM with proc_pidinfo
- [ ] 5B: stdout and stderr go to output.log, new text sent every 500 ms, remainder sent at the end, dir cleaned after final report
- [ ] 5C: cancel of a RUNNING job uses SIGKILL and waitpid, no zombies, cores and RAM freed
- [ ] 5D: current RAM, page faults and CPU time while running; peak RAM when finished
- [ ] 5E: diagnosis for normal exit, non-zero exit, 127, signals (SIGSEGV, SIGFPE and others); own kills report the reason, not a signal
- [ ] 9: `drain` on stdin and Ctrl+C both drain; jobs finish; final output sent; clean exit; no rescue
- [ ] 12: `-every` registers a template (ids from 1), first copy immediately, about 1 s timer, event 'Cron 1 injected Job 7 (run #3)', `crons`, `cancel-cron` with clear errors, cannot combine with `-depends-on`

- [ ] 14: OSN_CHECKPOINT_DIR, 500 ms stat polling, checkpoint sent and stored, sent again before binary on every restart, `migrate` (SIGTERM, then SIGKILL after 2 s, back to QUEUED with no attempt used, old worker excluded if another qualifies), `status` shows checkpoint number, size and age, clear errors for unknown or non-RUNNING jobs

8.6 Person B checklist

- [ ] `./scheduler <port> <FIFO|PRIORITY> [-discovery-port]` with the startup messages from the brief
- [ ] One concurrency model chosen, written up in the README, and nothing blocks anything else
- [ ] Queue capacity 1024 and QUEUE_FULL on submit; re-queued jobs are never dropped
- [ ] `status` and `nodes` never wait behind a scheduling pass
- [ ] Pass runs on submit, job finish, worker available, heartbeat, and at least once per second
- [ ] FIFO: the oldest job blocks the pass. PRIORITY: effective priority, 10 s aging, floor 1, older first on ties, moves on to the next job
- [ ] The pass follows the 'Order of Scheduling Decisions' page exactly
- [ ] Worker states: ALIVE, SUSPECTED after 1 missed heartbeat, DEAD after 3 (about 6 s), back to ALIVE on a heartbeat; only ALIVE workers get jobs; DRAINING excluded
- [ ] DEAD worker: RUNNING to LOST to QUEUED, retry limit as a constant, FAILED_PERMANENT and a client event after the last attempt; owners get SUSPECTED and DEAD warnings
- [ ] 4D: load-aware choice, logged as 'chosen over … (CPU a% vs b%)'
- [ ] Scheduler-side cancel for QUEUED, RUNNING and WAITING jobs
- [ ] 10: preempt only at step 4 for priority 1, victims lowest priority first then most recent, no attempt used, resources held for the priority 1 job, hold released if that job is cancelled or fails
- [ ] Tie-break order: idle, then cached binary, then lowest CPU, then fewest jobs

8.7 Person C checklist

- [ ] `./client [<ip> <port>] [-user] [-discovery-port]`; discovery first, fallback second
- [ ] Username 1 to 31 characters of letters, digits, `_`, `-`, `.`; asks again if invalid; sent first; re-sent after reconnect; two clients with the same name both accepted
- [ ] [Event] lines appear while idle at the prompt (poll on stdin and the socket)
- [ ] `nodes`, `submit`, `status`, `logs`, `logs -f` (Ctrl+C returns to the prompt), `cancel`

- [ ] `-cores -ram -time -prio` required, `-gpu` optional (default 0); later flags parsed and stored
- [ ] QUEUE_FULL shown to the user
- [ ] 2C: UDP responder (SO_REUSEADDR and SO_REUSEPORT, port 9999 default), finder waits 2 s and retries 3 times, works on Linux and macOS
- [ ] 7: log_line with [HH:MM:SS], uppercase tags; every worker state change, job state change, scheduling decision and discovery outcome is logged
- [ ] 6: at least 64 KB of output per job kept in the scheduler; plain `logs` works after the job ended; `-f` streams live and ends with a footer like '-- Job 5 finished --'
- [ ] 8: `-depends-on 1,2,3` and `g<id>` groups resolved at submit time; unknown ids rejected; WAITING to QUEUED when all COMPLETED; cascade CANCELLED when any dependency FAILED, FAILED_PERMANENT or CANCELLED
- [ ] 11: per-user running cores; skip when a user holds more than 50% of non-DEAD cores and another user who is not over quota has a QUEUED job; exactly 50% is fine; each skip logged once per pair per pass
- [ ] 13: SHA-256 or SHA-1 of the file content; worker cache/ folder named by file name and hash; OSN_CACHE_DIR; scheduler remembers hashes per worker; full file the first time, short 'already cached' message after; PATH commands never transfer; unreadable file runs by path
- [ ] 13 size limit: OSN_CACHE_SIZE_MB (default 512), least recently used evicted first, never the just-stored file or one in use; scheduler told after eviction; restart rescans and reports; missing file recovered without using an attempt
- [ ] 13 tests from the brief: same file twice, one byte changed, cached file deleted, two idle workers, and the A, B, A, C eviction scenario with a 2 MB cache

8.8 Gate checklists (run together)

Gate 0: Contract

- [ ] Repo created, all three added, git name and email correct everywhere
- [ ] Enums, job struct, worker struct, constants, message list, hooks table written
- [ ] Stubs compile; `make` builds ./scheduler ./worker ./client
- [ ] Roles A, B, C assigned; mocks folders created

Gate 1: Core pipeline (Milestone 1)

- [ ] Scheduler starts in FIFO and PRIORITY; worker finds it by UDP and by fallback; client logs in
- [ ] `nodes` shows the worker; submit runs a job under sudo with limits applied

- [ ] Output reaches the worker terminal; `status` shows COMPLETED or FAILED; events arrive while idle
- [ ] Every timestamped log line looks right
- [ ] 5-minute walkthrough from each person

Gate 2: Resilience and telemetry (Milestone 2)

- [ ] Kill a worker mid-job: SUSPECTED, DEAD, job re-queued on another worker, FAILED_PERMANENT after 3 attempts
- [ ] Cancel works for QUEUED and RUNNING; no zombies (check with ps)
- [ ] A memory-leaking job shows rising RAM in `status`
- [ ] Crash diagnosis shown for segfault, divide by zero, command not found, wall-clock timeout
- [ ] `logs` and `logs -f` work; draining works by `drain` and by Ctrl+C
- [ ] Load-aware decisions logged
- [ ] Rebalance check done

Gate 3: Advanced scheduling (Milestone 3)

- [ ] Dependency chain with cascade cancel; groups
- [ ] Preemption scenario from the brief; preempted job is not tried again until the priority 1 job starts
- [ ] Quota example on a 4-core worker; skip logged once
- [ ] Cron template runs repeatedly; `crons` and `cancel-cron` work
- [ ] All caching tests from the checklist above

Gate 4: Checkpoints and migration (Milestone 4)

- [ ] A job that checkpoints continues from its checkpoint after `migrate`
- [ ] Killing the worker gives the same result after the dead-worker timeout
- [ ] Preemption also resumes from the last checkpoint
- [ ] Full run through the 'Order of Scheduling Decisions' page

8.9 Final submission checklist

- [ ] Fresh clone builds with `make`; executables named exactly scheduler, worker, client
- [ ] Runs on Linux with sudo and on macOS, without Docker
- [ ] Every person has clearly attributed commits with proper messages
- [ ] No system() calls; no AI agent tools used on the code
- [ ] No giant main.c; shared code is in common/, not copy-pasted

- [ ] README covers build, run, concurrency model choice, protocol summary, who did what
- [ ] All gate checklists ticked
- [ ] Everyone can explain the whole system
- [ ] Submitted before 2 December 2026